This is a comprehensive Product Requirements Document (PRD) and Technical Architecture guide for **Micro-CFO**, designed to translate the vision in your PDF into a buildable, market-ready product.

## Part 1: Product Requirements Document (PRD)

### 1. Product Vision

To build an autonomous "Growth & Compliance" engine for Indian MSMEs that moves fintech from *descriptive* (dashboards) to *prescriptive* (active management), combining defense (audit protection) with offense (subsidy hunting).

### 2. Target Audience

- **Primary:** Tier-2/3 Manufacturers, Proprietors.
- **Secondary:** Gig-Economy Freelancers.
- **Key Pain Point:** "Compliance Fog"—fear of penalties (GST/MCA) and ignorance of benefits (Government Subsidies).

### 3. Core Feature Specifications (The "Quad-Agent" System)

- **Feature 1: The Visual Auditor (Defense)**
  - *Input:* Image of physical bill/invoice (via Camera/WhatsApp).
  - *Action:* Extract text, classify expense type (Personal vs. Business), validate against GST rules.
  - *Output:* "Safe to Claim" or "Warning: Personal Expense."
- **Feature 2: The Legislative Sentinel (Defense)**
  - *Input:* User business profile (sector, turnover).
  - *Action:* Continuous monitoring of MCA/RBI/GST circulars via RAG.
  - *Output:* Proactive alerts (e.g., "New MSME rule passed yesterday affects your specific filing deadline").
- **Feature 3: The Subsidy Hunter (Offense)**
  - *Input:* Transaction history + Business Assets data.
  - *Action:* Semantic match against Government Scheme databases (Startup India, PLI).
  - *Output:* "You bought machinery; you are eligible for the PMFME subsidy. Apply now?"
- **Feature 4: The Negotiator (Offense)**
  - *Input:* Cash flow prediction indicating a dip.
  - *Action:* Draft emails to vendors/clients.
  - *Output:* Auto-generated email drafts for credit extension or overdue payments.

---

## Part 2: Technical Architecture & Orchestration

To make this professional and scalable, we will move away from a simple script to a **Modular Agentic Workflow**.

### 1. High-Level Architecture

- **Frontend:** WhatsApp Interface (Twilio/Meta API) + Web Dashboard (React/Next.js).
- **Orchestrator:** A central "Brain" (FastAPI + LangGraph/LangChain) that routes tasks to specific agents.
- **Memory:** PostgreSQL (User Data/Logs) + Redis (Session/Conversation History).
- **Knowledge Base:** Vector Database (Pinecone/Milvus) for RAG.

### 2. Agent Orchestration (The "Flow")

We need a **Router Architecture**. When a user sends a message/image, the Router decides which Agent handles it.

- **Step 1: Ingestion.** User uploads an image or asks a question.
- **Step 2: Classification (The Router).** A lightweight LLM classifies the intent:
  - *Is it an image?* $\rightarrow$ Send to **Visual Auditor**.
  - *Is it a question about laws?* $\rightarrow$ Send to **Legislative Sentinel**.
  - *Is it about money/grants?* $\rightarrow$ Send to **Subsidy Hunter**.
- **Step 3: Execution.** The specific agent performs its task (OCR, Vector Search, or Drafting).
- **Step 4: Synthesis.** The response is formatted back into natural language and sent to the user.

### 3. RAG Implementation (Crucial for Sentinel & Hunter)

Since laws change, you cannot hard-code them. You need **Retrieval-Augmented Generation (RAG)**.

- **Data Ingestion Pipeline:**
  1. **Scraping:** Automated scripts fetch PDFs from mca.gov.in and rbi.org.in.
  2. **Chunking:** Do not just split by pages. Use **Semantic Chunking**. Group text by "Sections" or "Articles" of the law so context isn't lost.
  3. **Metadata Tagging:** Tag every chunk with metadata: { "source": "GST Act", "year": "2024", "sector": "manufacturing" }. This allows the AI to filter answers (e.g., "Only search laws relevant to Manufacturing").
  4. **Embedding:** Use an embedding model (like text-embedding-3-small or open-source bge-m3) to convert text to vectors.
  5. **Storage:** Store in a Vector DB (ChromaDB for local dev, Pinecone for production).

---

## Part 3: Professional UI/UX Design Strategy

To look "professional" without a designer, strictly follow **"Trust-First" Design Principles**.

#### 1. Color Palette (Psychology of Finance)

- Avoid flashy colors. Use **Deep Navy Blue (#0F172A)** for authority and **Emerald Green**

**(#10B981)** for success/growth.
- Use purely white backgrounds for content cards to ensure readability (high contrast).

**2. The Web Dashboard (The "Control Center")**

- **Layout:** Sidebar navigation (Audit, Compliance, Subsidies).
- **Typography:** Use clean, sans-serif fonts. **Inter** or **Roboto** are industry standards for fintech.
- **Data Visualization:** Do not use complex 3D charts. Use simple **Line Charts** for cash flow and **Progress Bars** for Subsidy Eligibility.
- **"Human" Touch:** The AI responses should appear in a chat interface on the right side of the screen, distinct from the hard data.

**3. WhatsApp Interface (The "Action Center")**

- Keep it conversational but concise.
- Use rich media: Don't just send text. Send **PDF Summaries** or **Buttons** ("Apply Now", "Ignore").

---

## Part 4: Backend Flow & Security

**1. The Backend Stack**

- **Language:** Python (best for AI/Data).
- **Framework: FastAPI**. It is faster than Flask and auto-generates documentation (Swagger UI), which makes you look professional to investors/partners.
- **Database:** PostgreSQL (Supabase is a great managed option) for user data, transaction logs, and auth.

**2. Security (Non-Negotiable for Fintech)**

- **Data Encryption:** Encrypt all sensitive columns (Bank Balance, GSTIN) in the database using AES-256.
- **PII Redaction:** Before sending any invoice text to an LLM (like GPT-4), run a "Scrubber" script to replace names/phone numbers with placeholders (e.g., [CLIENT_NAME]) to protect privacy.
- **Auth:** Use **JWT (JSON Web Tokens)**. Even for the MVP, never store passwords as plain text; use hashing (bcrypt).

---

## Part 5: Roadmap to "Market Ready"

**Phase 1: The "Lab" (Weeks 1-3)**

- **Goal:** Build the "Visual Auditor" only.
- **Action:**
  - Set up FastAPI.

    - Integrate Google Gemini Flash (cheaper/faster for vision) to read invoices.
    - Prompt engineering: "Extract Date, Total, GSTIN. Classify as Business/Personal."
    - *Result:* A script that takes an image and outputs JSON.

**Phase 2: The "Brain" & RAG (Weeks 4-6)**

- **Goal:** Build the "Legislative Sentinel."
- **Action:**
    - Scrape 50 key PDFs (GST Act, recent circulars).
    - Set up the Vector DB.
    - Build the retrieval chain: Query -> Search DB -> LLM Answer.
    - *Result:* A chatbot that answers "What is the penalty for late GST filing?" accurately citing sources.

**Phase 3: The Interface (Weeks 7-9)**

- **Goal:** Connect to WhatsApp.
- **Action:**
    - Use Twilio Sandbox or WhatsApp Business API.
    - Connect the FastAPI backend to the WhatsApp webhook.
    - *Result:* You send a photo to the bot; it replies with the audit result.

**Phase 4: Testing & Polish (Weeks 10-12)**

- **Goal:** False Acceptance Rate (FAR) Testing.
- **Action:**
    - Run the "Audit Challenge" mentioned in your PDF (500 mixed invoices).
    - Refine prompts based on failures.
    - Deploy the Web Dashboard (Vercel/Netlify for frontend, Render/Railway for backend).